

Documentação da rota

/api/garantia

Visão geral

A rota `/api/garantia` gerencia garantias vinculadas a uma **ordem de serviço original**.

Ela permite:

- Listar garantias cadastradas.
- Buscar dados auxiliares para o front, como OS, clientes, vendedores e lentes.
- Criar uma nova garantia baseada em uma OS existente.
- Atualizar uma garantia.
- Excluir uma garantia, somente para usuários administradores.

A rota usa Supabase e exige que o usuário esteja autenticado e com perfil ativo.

Autenticação e segurança

Todas as operações passam pela função interna:

```
getAuthenticatedProfile()
```

Ela valida:

1. Se existe usuário autenticado no Supabase Auth.
2. Se existe um perfil correspondente na tabela `usuarios`.
3. Se o usuário está com `status = "ativo"`.

Possíveis erros de autenticação

Situação	St	Resposta
	;	
	t	
	l	
	s	

Usuário não autenticado	401	Usuário não autenticado.
Perfil não encontrado	404	Perfil de usuário não encontrado.
Usuário inativo/bloqueado	403	Seu acesso está inativo ou bloqueado.

GET /api/garantia

Busca todas as garantias da conta do usuário e também dados auxiliares usados pelo front.

Descrição

Essa rota retorna:

- Garantias.
- Ordens de serviço.
- Clientes.
- Vendedores.
- Lentes.
- Perfil do usuário autenticado.

Os filtros ficam no front. A API apenas retorna os dados da conta logada.

Regras

A busca sempre filtra por:

```
.eq("conta_id", profile.conta_id)
```

Ou seja, o usuário só enxerga dados da própria conta.

Resposta de sucesso

```
{
```

```

"success": true,

"garantias": [],

"ordensServico": [],

"clientes": [],

"vendedores": [],

"lentes": [],

"user": {

  "id": "uuid",

  "conta_id": "uuid",

  "role": "admin",

  "status": "ativo"

}

}

```

Possíveis erros

Situação	Status	Mensagem
Erro ao buscar garantias	400	Não foi possível buscar as garantias.
Erro ao buscar ordens de serviço	400	Não foi possível buscar as ordens de serviço.
Erro ao buscar clientes	400	Não foi possível buscar os clientes.

Erro ao buscar vendedores	40	(Não foi possível buscar os vendedores.
Erro ao buscar lentes	40	(Não foi possível buscar as lentes.
Erro interno	50	(Erro interno ao buscar garantias.

POST /api/garantia

Cria uma nova garantia vinculada a uma OS existente.

Descrição

Essa rota recebe os dados da garantia, valida a OS original, copia informações importantes da OS e da lente original, e cria um registro na tabela `garantias`.

Body aceito

A rota aceita o payload de duas formas:

Forma direta

```
{  
  "os_id": "uuid-da-os",  
  "motivo_garantia": "Lente apresentou defeito",  
  "status_garantia": "aberta",  
  "custo_garantia": 80,  
  "valor_cobrado_cliente": 0  
}
```

Forma aninhada

```
{
  "garantia": {
    "os_id": "uuid-da-os",
    "motivo_garantia": "Lente apresentou defeito",
    "status_garantia": "aberta",
    "custo_garantia": 80,
    "valor_cobrado_cliente": 0
  }
}
```

A função que permite isso é:

```
getGarantiaSegment(body)
```

Ela usa `body.garantia` quando existir, senão usa o próprio `body`.

Campos obrigatórios

Na prática, o campo mais importante é:

```
{
  "os_id": "uuid-da-os-original"
}
```

Sem ele, a API retorna:

```
{
  "error": "Selecione a OS original da garantia."
}
```

com status `400`.

O que a API faz automaticamente

Ao criar uma garantia, a rota:

1. Remove campos proibidos do payload

A função `cleanPayload()` remove campos como:

`conta_id`
`created_at`
`updated_at`
`created_by_admin_id`
`updated_by_admin_id`
`responsavel_abertura_id`
`responsavel_finalizacao_id`

Também remove objetos aninhados como:

`garantia`
`ordemServico`
`ordem_servico`
`os`
`cliente`
`vendedor`
`lente`
`lentes`
`armacao`
`anexos`
`documentos`

Isso evita que o front envie lixo ou sobrescreva dados sensíveis.

2. Normaliza campos vazios

Strings vazias viram `null`.

Exemplo:

```
{
```

```
"observacao": ""
```

```
}
```

vira:

```
{
```

```
"observacao": null
```

```
}
```

3. Normaliza valores financeiros

Os campos abaixo são convertidos para número com duas casas:

custo_garantia

valor_cobrado_cliente

Exemplo:

```
{
```

```
"custo_garantia": "R$ 120,50"
```

```
}
```

vira:

```
{
```

```
"custo_garantia": 120.5
```

```
}
```

4. Busca a OS original

A rota verifica se a OS existe e pertence à mesma conta do usuário.

Se não encontrar:

```
{
```

```
"error": "OS original não encontrada nesta conta."
```

```
}
```

Status: 404.

5. Busca a lente original

A rota tenta buscar a lente original de duas formas:

- Se `lente_original_id` for enviado, busca por esse ID.
- Se não for enviado, pega a primeira lente vinculada à OS original.

6. Cria um snapshot da OS original

A garantia recebe automaticamente dados da OS original, como:

`os_id`

`cliente_id`

`vendedor_id`

`numero_os_original`

`numero_nf`

`pedido_laboratorio_numero`

`laboratorio_nome`

`telefone_laboratorio`

E, se houver lente original:

`lente_original_id`

`lente_tipo`

`lente_marca`

`lente_linha`

`lente_laboratorio`

`lente_material`

`lente_indice_refracao`

`tratamento_antirreflexo`

`tratamento_filtro_azul`

`tratamento_fotossensivel`

```
tratamento_polarizado
```

```
tratamento_uv
```

```
tratamento_risco
```

Essa parte é importante porque a garantia fica com um “congelamento” dos dados principais da OS e da lente. Boa decisão de arquitetura, inclusive. Evita que mudanças futuras na OS baguncem o histórico da garantia.

7. Define status padrão

Se `status_garantia` não for enviado, a API define:

```
status_garantia = "aberta"
```

8. Define data de abertura

Se `data_abertura` não for enviada, define a data atual:

```
data_abertura = hoje
```

9. Salva responsáveis

A API grava automaticamente:

```
responsavel_abertura_id: profile.id
```

```
created_by_admin_id: profile.id
```

```
updated_by_admin_id: profile.id
```

Resposta de sucesso

Status: `201`

```
{  
  
  "success": true,  
  
  "message": "Garantia cadastrada com sucesso.",  
  
  "garantia": {
```

```

    "id": "uuid",
    "os_id": "uuid",
    "cliente_id": "uuid",
    "status_garantia": "aberta"
  }
}

```

Possíveis erros

Situação	Status	Mensagem
	400	Selecione a OS original da garantia.
OS não encontrada	404	OS original não encontrada nesta conta.
Cliente não informado	400	Cliente da garantia não informado.
Cliente não encontrado	404	Cliente não encontrado nesta conta.
Erro ao inserir	400	Não foi possível cadastrar a garantia.
Erro interno	500	Erro interno ao cadastrar garantia.

PUT /api/garantia

Atualiza uma garantia existente.

Descrição

A rota atualiza uma garantia já cadastrada, validando se ela pertence à conta do usuário logado.

Body aceito

Também aceita payload direto ou aninhado.

Forma direta

```
{  
  "id": "uuid-da-garantia",  
  "status_garantia": "resolvida",  
  "observacao": "Garantia resolvida com troca da lente.",  
  "custo_garantia": 100,  
  "valor_cobrado_cliente": 0  
}
```

Forma aninhada

```
{  
  "garantia": {  
    "id": "uuid-da-garantia",  
    "status_garantia": "resolvida",  
    "observacao": "Garantia resolvida com troca da lente."  
  }  
}
```

Também aceita:

```
{  
  "garantia_id": "uuid-da-garantia"  
}
```

Campo obrigatório

Para atualizar, precisa informar o ID da garantia.

Se não enviar:

```
{  
  "error": "ID da garantia não informado."  
}
```

Status: 400.

Regras de atualização

Antes de atualizar, a rota:

1. Busca a garantia existente.
 2. Confirma que ela pertence à mesma `conta_id`.
 3. Prepara o payload.
 4. Reaplica snapshot da OS original.
 5. Valida o cliente.
 6. Atualiza responsável de finalização quando necessário.
-

Status finais

A rota considera como status finais:

```
["resolvida", "cancelada", "recusada"]
```

Quando a garantia muda para um desses status, a API pode preencher:

`data_finalizacao`

`responsavel_finalizacao_id`

Datas automáticas por status

A função `applyStatusDates()` define datas automaticamente quando o status muda.

Quando status vira `enviada_laboratorio`

Se `data_envio_laboratorio` não existir, define a data atual.

```
status_garantia = "enviada_laboratorio"
```

Quando status vira `aguardando_retorno`

Também define `data_envio_laboratorio`, caso ainda não exista.

```
status_garantia = "aguardando_retorno"
```

Quando status vira final

Para:

`resolvida`

`cancelada`

`recusada`

define:

```
data_finalizacao = hoje
```

Resposta de sucesso

```
{  
  "success": true,  
  "message": "Garantia atualizada com sucesso."  
}
```

```
"garantia": {  
  "id": "uuid",  
  "status_garantia": "resolvida",  
  "data_finalizacao": "2026-05-21"  
}  
}
```

Possíveis erros

Situação	Status	Mensagem
	400	Garantia não encontrada.
ID não informado	400	ID da garantia não informado.
Garantia não encontrada	400	Garantia não encontrada.
OS original não encontrada	400	OS original não encontrada nesta conta.
Cliente não informado	400	Cliente da garantia não informado.
Cliente não encontrado	400	Cliente não encontrado nesta conta.
Erro ao atualizar	400	Não foi possível atualizar a garantia.

Erro interno

50

```
{  
  "error": "Erro interno ao atualizar  
  garantia."  
}
```

DELETE /api/garantia

Exclui uma garantia.

Descrição

Remove uma garantia da tabela `garantias`.

Permissão

Somente usuários com:

```
role = "admin"
```

podem excluir garantias.

Se o usuário não for admin:

```
{  
  
  "error": "Apenas administradores podem excluir garantias."  
}
```

Status: `403`.

Body

```
{  
  
  "id": "uuid-da-garantia"  
}
```

Fluxo

A rota:

1. Valida autenticação.
2. Verifica se o usuário é admin.
3. Valida se o ID foi enviado.
4. Confirma se a garantia existe na conta do usuário.
5. Exclui a garantia.

Resposta de sucesso

```
{  
  "success": true,  
  "message": "Garantia excluída com sucesso."  
}
```

Possíveis erros

Situação	Status	Mensagem
	200	
Usuário não é admin	403	Apenas administradores podem excluir garantias.
ID não informado	400	ID da garantia não informado.
Garantia não encontrada	404	Garantia não encontrada.
Erro ao excluir	400	Não foi possível excluir garantia.

Erro interno

50

Erro interno ao excluir garantia.

(

Campos importantes da garantia

Com base na rota, os principais campos manipulados são:

id

conta_id

os_id

cliente_id

vendedor_id

lente_original_id

status_garantia

data_abertura

data_envio_laboratorio

data_finalizacao

numero_os_original

numero_nf

pedido_laboratorio_numero

laboratorio_nome

telefone_laboratorio

lente_tipo

lente_marca

lente_linha

lente_laboratorio

lente_material

lente_indice_refracao

tratamento_antirreflexo

tratamento_filtro_azul

tratamento_fotossensivel

tratamento_polarizado

tratamento_uv

tratamento_risco

custo_garantia

valor_cobrado_cliente

responsavel_abertura_id

responsavel_finalizacao_id

created_by_admin_id

updated_by_admin_id

created_at

updated_at

Exemplo de uso no front

Buscar dados

```
const response = await fetch("/api/garantia");

const data = await response.json();


console.log(data.garantias);

console.log(data.ordensServico);

console.log(data.clientes);

console.log(data.vendedores);

console.log(data.lentes);
```

Criar garantia

```
const response = await fetch("/api/garantia", {

  method: "POST",

  headers: {

    "Content-Type": "application/json",

  },

  body: JSON.stringify({

    garantia: {

      os_id: "uuid-da-os",

      motivo_garantia: "Lente com defeito de fabricação",

      status_garantia: "aberta",

      custo_garantia: 0,

      valor_cobrado_cliente: 0,

    },

  }),

});
```

```
const data = await response.json();
```

Atualizar garantia

```
const response = await fetch("/api/garantia", {  
  method: "PUT",  
  headers: {  
    "Content-Type": "application/json",  
  },  
  body: JSON.stringify({  
    garantia: {  
      id: "uuid-da-garantia",  
      status_garantia: "resolvida",  
      observacao: "Cliente recebeu nova lente.",  
    },  
  }),  
});
```

```
const data = await response.json();
```

Excluir garantia

```
const response = await fetch("/api/garantia", {  
  method: "DELETE",  
  headers: {  
    "Content-Type": "application/json",  
  },  
  body: JSON.stringify({
```

```
id: "uuid-da-garantia",
```

```
}),
```

```
});
```

```
const data = await response.json();
```

```
•
```